Project 1

Implementing CFS scheduling

Due date
2026. 04. 24. 23:59

Overview

- **Goal**
  - Replace xv6's default Round Robin scheduler with a simplified version of Linux's **Completely Fair Scheduler (CFS)**.
- **Submission Guidelines**
  - Submit the modified **xv6 source code** and the **report** for this assignment.
  - Push all your source code and your report to the following GitHub Classroom repository's **main branch**:
    - https://classroom.github.com/a/Z6qG6k3F
- **Deadline: April 24, 2026 (Wed) 23:59**
  - Late submissions are allowed, but your final score will be reduced by **5%** for each **3-hour window** following the due date.
  - Grading is based on your **final commit**. Be extremely cautious about committing after the deadline, as it will trigger the late penalty.

- There will be a **short quiz** right before the midterm exam on **April 25, 2026 (Sat)**.

DO NOT ASK THIS ISSUE AGAIN

GitHub Classroom   GitHub Education

Repository Access Issue

You no longer have access to your assignment repository. Contact your teacher for support

Your assignment is due by **Apr 1, 2026, 14:59 UTC**

Join the GitHub Student Developer Pack

Verified students receive free GitHub Pro plus thousands of dollars worth of the best real-world tools and training from GitHub Education partners — for free. For more information, visit GitHub Student Developer Pack.

Apply

Discussions

- ↑ 1 [Assignment 1] <레포지토리 접근 실패> — WONHO-JEONG-HANYANG asked yesterday in Questions · Closed · Unanswered — 2
- ↑ 1 [assignment1] Assignment 1 레포지토리에 접근할 수가 없습니다. — yenru0 asked yesterday in Questions · Closed · Unanswered — 2
- ↑ 1 [assignment1] <레포지토리 접근 실패> — kwon120 asked 2 days ago in Questions · Closed · Answered — 2
- ↑ 1 [assignment1] assignment1 repository Access Issue 발생으로 인한 접근 불가 문의 — EUNCH asked 2 days ago in Questions · Closed · Answered — 2
- ↑ 1 assignment repository 접근이 불가능합니다 — yeeunkim97 asked 4 days ago in Questions · Closed · Answered — 4

ESeokhwan last week Maintainer — edited

이와 같이 메일을 못 찾겠는 경우에는 다음 형식의 url에 접속하여 invitation을 받아보시길 바랍니다.

`https://github.com/2026-HYU-ELE3021/{과제 이름}-{GitHub username}/invitations`

예시) https://github.com/2026-HYU-ELE3021/assignment1-donghyeonahn/invitations

Unmark as answer · ↑ 1 · 1 reply

wnsah814 2 days ago Maintainer — edited

classroom 수락 후 메일로 보내진 초대 링크를 1주일 내로 수락하지 않으면 유효기간이 만료되어 접속이 불가능합니다.

다시 초대링크를 생성하였으니 확인해보시길 바랍니다.

https://github.com/2026-HYU-ELE3021/assignment1-kwon120/invitations

Unmark as answer · ↑ 1 · 1 reply

Core Concept of CFS

- **Completely Fair Scheduler (CFS)**
  - A scheduling algorithm that ensures CPU time is allocated proportionally among processes based on their priority.
  - It achieves this by consistently selecting and executing the runnable process with the lowest **virtual runtime**.
- **Virtual Runtime**
  - A metric that tracks the total CPU execution time a process has been scheduled, scaled by its priority.
  - Each process has **its own** virtual runtime. The scheduler updates the running process's virtual runtime based on its actual runtime and its priority.
- **Nice Value**
  - A priority hint that determines how rapidly virtual runtime is accumulated.
  - A **higher** nice value causes the scheduler to increase the virtual runtime value at a **faster** rate. As a result, the process is selected for execution **less frequently**.

Specifications

- **Process Selection**
  - Select the `RUNNABLE` process with the lowest virtual runtime.
  - Utilize the provided **Red-Black tree** data structure to manage runnable processes efficiently.

- **Red-Black Tree**
  - A self-balancing binary search tree that guarantees $O(\log n)$ time complexity for insertions, deletions, and lookups.

```
$ vim kernel/rbtree.h

struct rb_node {
  uint64 key;
  void *data;
  struct rb_node *_parent;
  struct rb_node *_left;
  struct rb_node *_right;
  rb_color _color;
};

struct rb_tree {
  struct rb_node *root;
};

void rb_insert(struct rb_tree *tree, struct rb_node *node);
void rb_delete(struct rb_tree *tree, struct rb_node *node);

struct rb_node* rb_first(struct rb_node *node); // minimum
struct rb_node* rb_last(struct rb_node *node); // maximum
```

Specifications

- **Virtual Runtime Initialization & Update**
  - Every time a process is selected, its virtual runtime is increased based on its priority.
  - **Each increment of 1** in the nice value **doubles** the amount of virtual runtime added.
  - When a process is **forked**, initialize its virtual runtime to **its parent's virtual runtime**. If there is no parent, initialize it to `0`.
  - When a process wakes up from `SLEEPING`, update its virtual runtime according to the following rules, evaluated in order:
    - If there are any `RUNNABLE` processes, set the waking process's virtual runtime to the minimum virtual runtime among all `RUNNABLE` processes.
    - If there are no `RUNNABLE` processes, but at least one process is currently `RUNNING`, set the waking process's virtual runtime to the lowest virtual runtime of any `RUNNING` process.
    - If there are neither `RUNNABLE` nor `RUNNING` processes, set the waking process's virtual runtime to 0.

Specifications

- **Nice value Initialization**
  - The nice value must always be in the range from `-3` to `2`.
  - Initialize the nice value to `0` (default priority) for the very first process.
  - When a process is forked, it inherits its parent's nice value.

- **`set_nice()` system call**
  - **Role:** Sets the nice value (priority) of the currently running process.
  - **Signature:**

```
int set_nice(int nice);
```

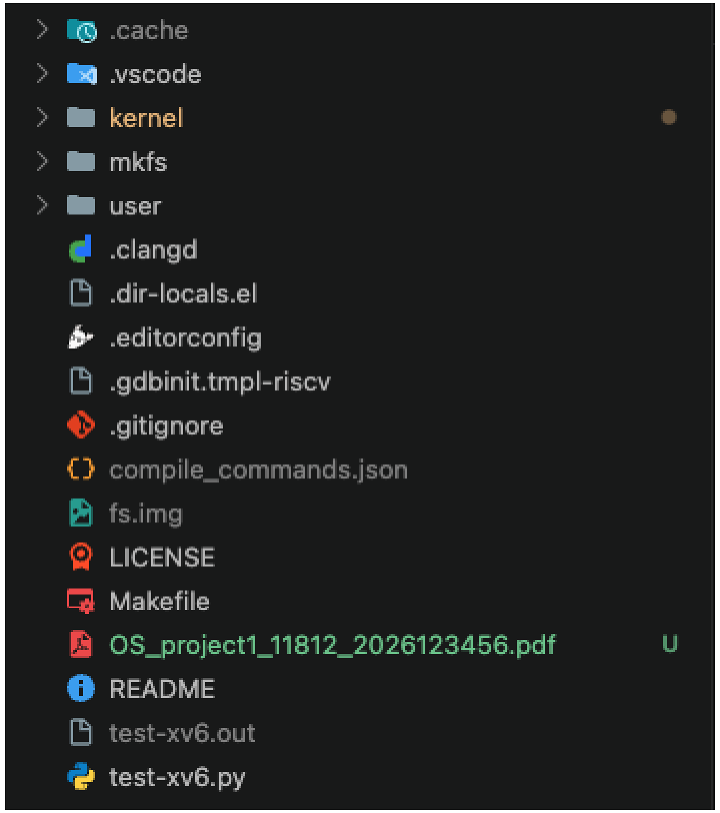
  - **nice (int):** The target nice value to apply. Must be within the valid range (from -3 to 2).
  - **Return value**:
    - Return 0 if the nice value is successfully updated.
    - Return 1 if the provided nice value is invalid.

Report

- **Design**
  - Explain your implementation plan to meet the assignment requirements.
- **Implementation**
  - Detail the specific modifications and additions you made to the given xv6 codebase. Clearly highlight how they differ from the original code and the purpose of these changes.
- **Results**
  - Document the compilation and execution process. Include screenshots verifying that all requirements are met, alongside explanations of the execution flow.
- **Troubleshooting**
  - If you encountered any issues during the assignment, describe the issues and your resolution process. For unresolved issues, explain what they were and what approaches you attempted.

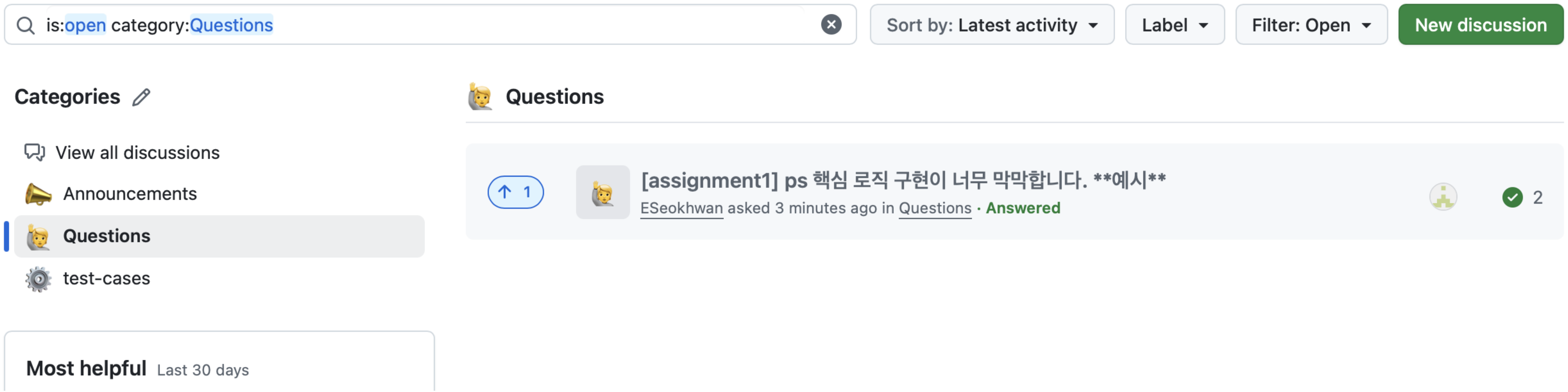
Report

- The report must be a PDF file strictly named: **OS_project1_[class number]_[student ID].pdf**
- You must commit and push your report PDF to your GitHub Classroom repository along with your xv6 source code.
- Make sure that report PDF is in the root directory of your repository, as shown in below.



Q&A

- For assignment related questions, please use the **Discussions tab** of xv6-sandbox repository.
  - https://github.com/2026-HYU-ELE3021/xv6-riscv-sandbox/discussions
- **Questions sent by email will not be answered.**
- For non-assignment questions or personal matters, please use an email like before.



- Students are highly **encouraged to answer each other's questions**! Active participation is greatly appreciated and helpful for everyone

Test Case Contribution

- You can share your custom test cases on the **Discussions tab** of xv6-sandbox repository too.
  - https://github.com/2026-HYU-ELE3021/xv6-riscv-sandbox/discussions
- A structured template is already provided, so we highly encourage your active contributions.
- Well-crafted test cases might actually be adopted for the official grading process!